

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Título del Proyecto:

C++ DISK ONLINE

PONDERACIÓN: 30 pts

 **Tiempo estimado: 168 hrs**

Índice

1. MARCO FORMATIVO	3
1.1. Valor.....	3
1.2. Competencia(s)	3
1.3. Habilidad(es) blandas a formar.....	3
2. Resultado del Aprendizaje	3
2.1. Objetivo SMART.....	3
3. Resumen Ejecutivo	4
4. Enunciado del Proyecto	4
4.1 Descripción del problema o necesidad a resolver.....	4
4.2 Alcance del proyecto	4
4.3 Recursos y herramientas a utilizar	4
4.4 Entregables	4
5. Material de apoyo	5
6. Metodología.....	5
7. Cronograma.....	6
8. Rúbrica de Calificación.....	6
8.1 Requisitos para optar a la calificación	6
8.2 Detalle de la Calificación	7
8.3 Comentarios Generales.....	8

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Honestidad	Este valor se aplica dentro del laboratorio mediante el desarrollo del proyecto de manera individual, se busca que cada estudiante implemente su propio código en base a la solución propuesta.

1.2. Competencia(s)

Con la elaboración de este proyecto usted adquirirá la siguiente competencia:

Analiza soluciones de almacenamiento en entornos híbridos mediante la comparación de sus características operativas y criterios de implementación para seleccionar alternativas que respondan a necesidades de eficiencia, escalabilidad y disponibilidad

1.3. Habilidad(es) blandas a formar

El proyecto le permitirá desarrollar las siguientes habilidades:

- Autogestión
- Resolución de problemas
- Pensamiento estratégico
- Pensamiento crítico
- Comunicación técnica

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Los estudiantes serán capaces de implementar y gestionar un sistema de archivos simulado, integrando backend en C++ y frontend	Lograr al menos un 80% de cumplimiento funcional en los comandos solicitados. 100% en lo solicitado en relación con la nube	Utilizar el lenguaje C++, frameworks modernos disponibles y los servicios de AWS (EC2 y S3)	Fortalecer habilidades prácticas en sistemas operativos, manejo de archivos, programación bajo nivel y despliegue en la	Cumplir este aprendizaje en un plazo máximo de 5 semanas.

web para la visualización y administración de discos utilizando AWS EC2 para el backend y S3 para el frontend			nube, brindando la base para proyectos de cursos más avanzados.	
---	--	--	---	--

3. Resumen Ejecutivo

El proyecto C++ DISK ONLINE consiste en la evolución del sistema de archivos simulado desarrollado previamente, incorporando una arquitectura web que permite ejecutar comandos, administrar discos y visualizar la estructura interna del sistema de archivos de forma gráfica.

La solución deberá contar con un backend desarrollado en C++, encargado de procesar comandos, administrar archivos binarios .mia, gestionar particiones, usuarios, permisos, estructuras EXT2/EXT3 y operaciones sobre carpetas y archivos. Además, deberá contar con un frontend web, desarrollado con un framework moderno, que permita ejecutar comandos, iniciar sesión, visualizar discos, particiones, carpetas y archivos.

El sistema deberá ser desplegado en la nube utilizando AWS EC2 para el backend y AWS S3 para el frontend. También se deberá incorporar soporte para el sistema de archivos EXT3, incluyendo una estructura de journaling que registre las operaciones realizadas por los usuarios dentro del sistema.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

Arquitectura

El proyecto tendrá la siguiente arquitectura:

- Frontend

La interfaz gráfica de usuario deberá ser desarrollada en una página web mediante un framework como React, Angular, Vue, u otro, dejando a discreción del estudiante el framework a utilizar.

Dicha página web deberá ser desplegada mediante el servicio de bucket S3 de AWS.

- Backend

Dicho backend deberá ser desplegado en una instancia EC2 de AWS. Esta instancia deberá tener como sistema operativo alguna distribución Linux, se recomienda Ubuntu.

La infraestructura final será:

Frontend -> S3 BUCKET

Backend -> EC2

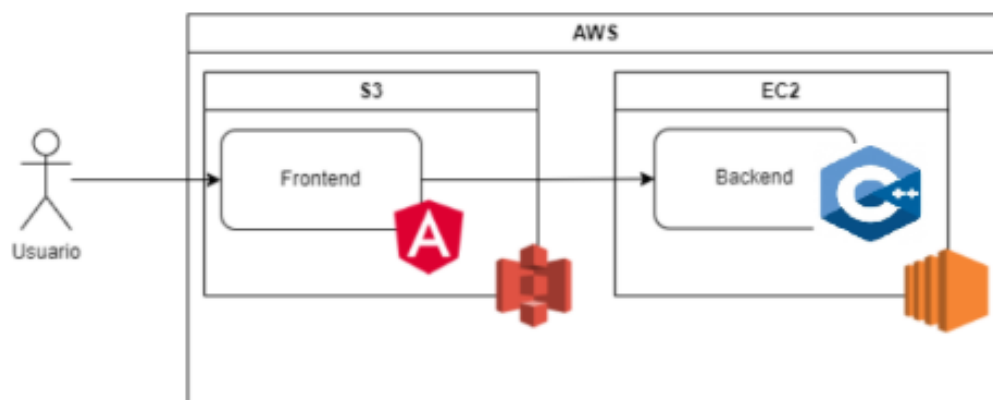


Figura 1 - Infraestructura en nube

Figura 1 - Infraestructura en nube

Primer Parte (Frontend)

Primero se mostrará la página principal del programa, la cual contará con una terminal para poder ejecutar comandos. Esta servirá para poder ejecutar todos los comandos iniciales (para la creación de discos, montaje y formateo de las particiones). Esta contará con un botón para poder iniciar sesión con alguno de los usuarios de una partición.

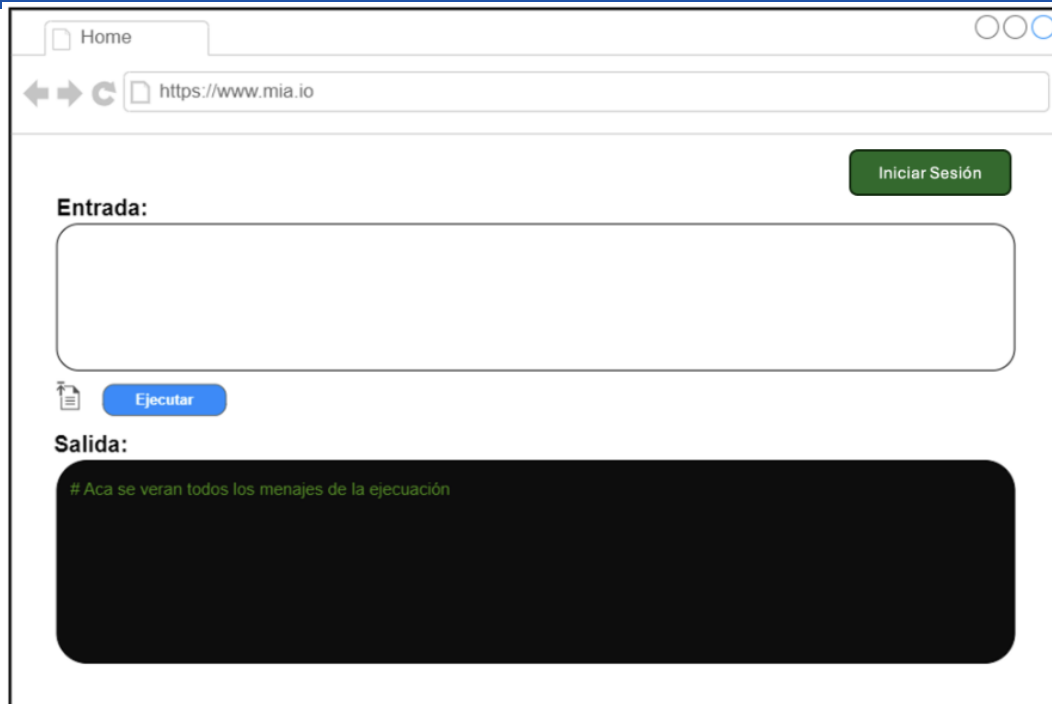


Figura 2 - Página Principal

Iniciar Sesión:

Se requiere una página para iniciar sesión ahora se realizará el login por medio de esta interfaz gráfica y no por comando, los comandos que requieren iniciar sesión previo a utilizarse son MKGRP, RMGRP, MKUSR, RMUSR y otros que usted considere necesarios.

A login form interface with a white background and a light gray border. At the top, the word 'Login' is centered in a bold, dark gray font. Below it, there are three input fields. The first is labeled 'ID Particion' and contains the text '341A'. The second is labeled 'Usuario:' and contains the text 'root'. The third is labeled 'Contraseña:' and contains a series of asterisks '*****'. Below these fields is a checkbox labeled 'Recordar usuario'. At the bottom of the form is a blue button with the word 'Submit' in white text.

Figura 3 - Página para iniciar sesión

Visualizador del Sistema de Archivos

Una vez logueado se ofrecerá una interfaz de usuario que permitirá explorar el sistema de archivos de manera visual, similar a un explorador de archivos, pero en modo solo lectura.

Los usuarios podrán examinar la estructura de directorios y archivos sin la capacidad de modificar, agregar o eliminar elementos. La interfaz está diseñada para facilitar una navegación intuitiva y clara, mostrando la jerarquía de discos, particiones y carpetas de forma organizada.

El proceso consta de los siguientes pasos:

1. Selección de disco

Se deberá poder seleccionar alguno de los discos creados mediante la siguiente vista:



Figura 4 - Página inicio del visualizador y selección del disco

Cada disco también deberá mostrar su información básica como:

- Capacidad del disco
- Tipo de fit
- Particiones montadas
- Fecha
- Entre otras

2. Selección de partición

Se deberá poder seleccionar alguna de las particiones del disco seleccionado:

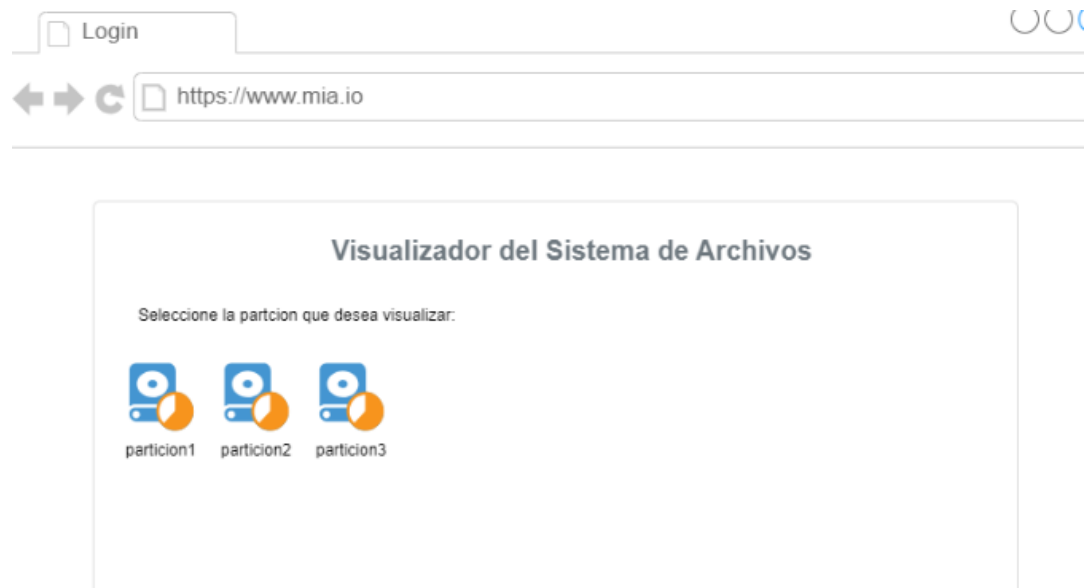


Figura 5 - Página selección de partición

Igualmente, cada una de las particiones deberá de mostrar su información básica, siendo **imprescindible que se muestre su tamaño**, fit y el estado de la misma.

3. Navegación con el sistema de archivos

Permite comenzar desde la carpeta raíz "/" cuando se utiliza el visualizador:



Figura 6 - carpeta root

Se deberá poder navegar por el sistema de archivos, donde la creación de archivos o carpetas será por medio de comandos en la página home y se podrán ver reflejados los cambios en esta pantalla. Tanto los archivos como las carpetas deben mostrar su información básica, incluyendo los permisos.

En esta pantalla, se podrá acceder a otras carpetas para navegar dentro de ellas.



Figura 7 - Visualizador del sistema de archivos actualizado

Dentro de esta interfaz, también se debe poder mostrar el contenido de los archivos.

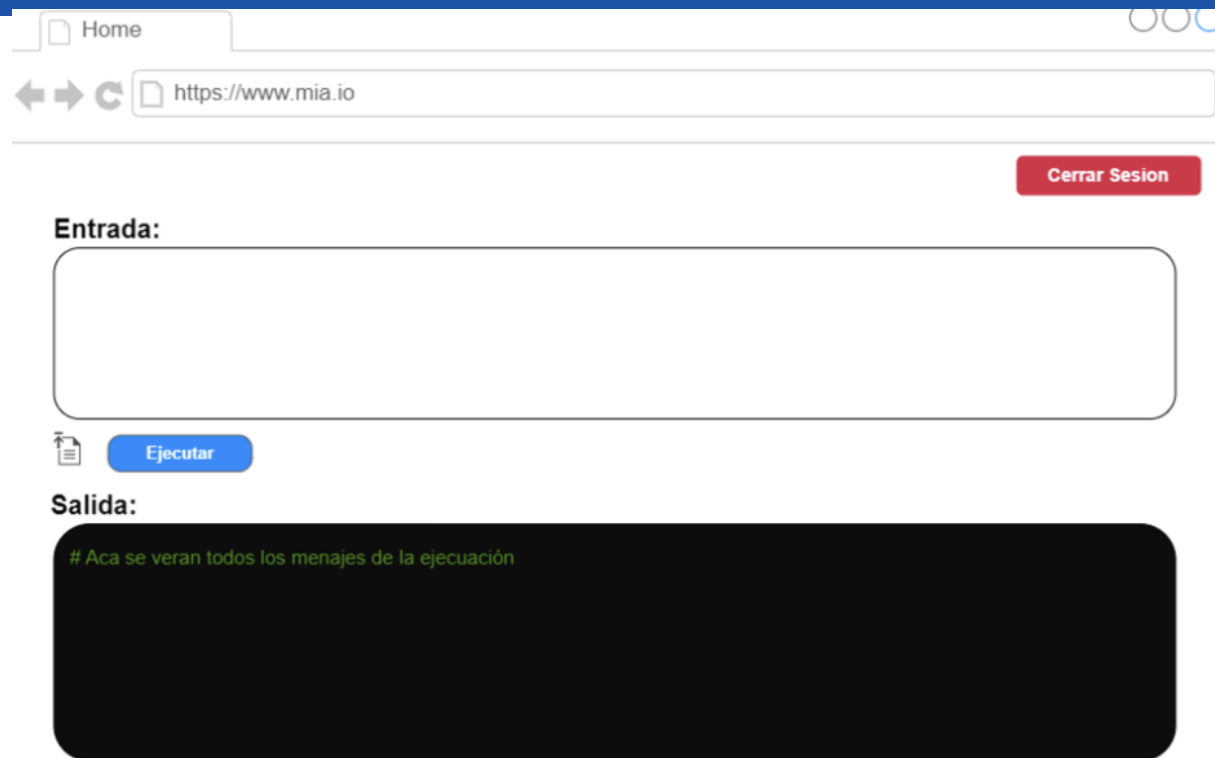


Figura 8

Visualizador de archivo de texto

Al estar logueado, en la pantalla principal se deberá tener un botón para cerrar sesión. La sesión activa servirá para ejecutar comandos en la terminal que requieran de una misma (por ejemplo cat, mkdir, mkfile, entre otros que verán más adelante).

- La idea principal es que el login siga funcionando como en el primer proyecto pero ahora, en lugar de comandos, se estará haciendo a través de la interfaz gráfica.



The screenshot shows a web application interface. At the top, there is a navigation bar with a "Home" link and a "Cerrar Sesion" button. Below the navigation bar, there is a large text input field labeled "Entrada:". To the right of the input field, there is a "Cerrar Sesion" button. Below the input field, there is a "Ejecutar" button. Below the "Ejecutar" button, there is a section labeled "Salida:" which contains a large black rectangular area. Above this black area, there is a green text label that reads "# Aca se veran todos los mensajes de la ejecución".

Home

https://www.mia.io

Cerrar Sesion

Entrada:

Ejecutar

Salida:

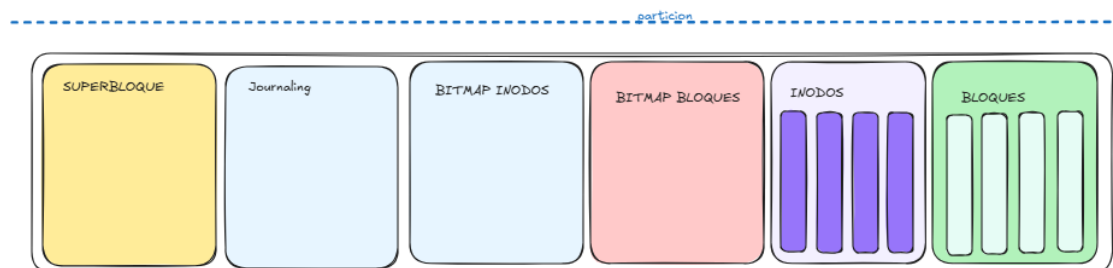
Aca se veran todos los mensajes de la ejecución

Figura 9 - Cerrar Sesión activa

Se agrega el manejo de Sistema de Archivos Ext3

EXT3

Para el caso del sistema de archivos EXT3, se deberán implementar las estructuras como se especifican a continuación. La estructura en bloques es la siguiente:



El número de bloques será el triple que el número de inodos. El número de inodos y bloques a crear se puede calcular despejando n de la primera ecuación y aplicando la función floor al resultado:

- $\text{tamaño_particion} = \text{sizeof}(\text{superblock}) + n * \text{sizeof}(\text{Journaling}) + n + 3 * n + n * \text{sizeof}(\text{inodos}) + 3 * n * \text{sizeof}(\text{block})$
- $\text{numero_estructuras} = \text{floor}(n)$

Observaciones:

- sizeof es el tamaño de los Structs.
- En el Bitmap de Bloques y Bloques se multiplica por tres debido a que existen tres tipos de bloque que son: bloques carpetas, bloques archivos y bloques de contenido.
- El valor del Journaling se va a manejar con una constante de 50.

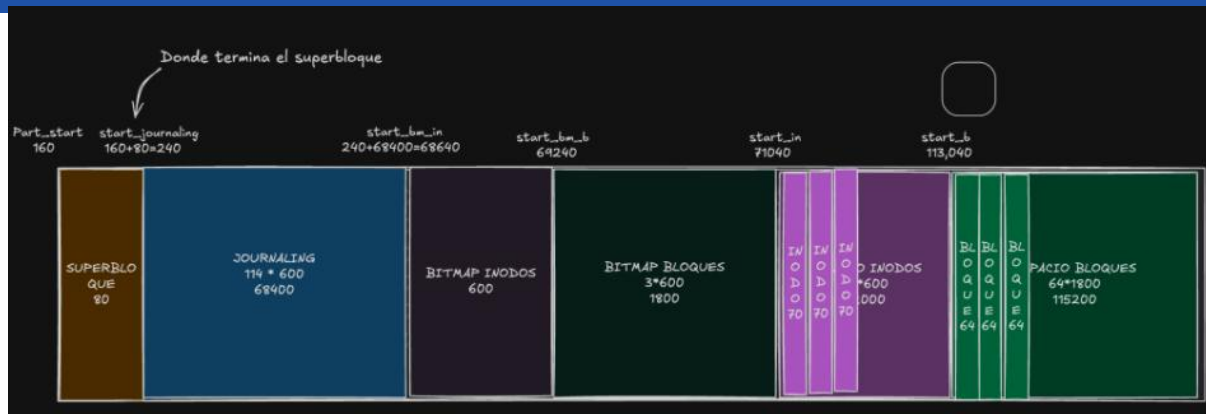


Figura 9 - Espacio de la partición 1

En este caso se tomó la estructura de bloques del sistema EXT2 que en este ejemplo se toma como si se hubiera asignado a la partición 1 con dicho sistema, **se aclara** que los bloques de Inodos y Bloques solo son reserva de espacio, ya que dentro de ese espacio se escribirán múltiples estructuras contiguas según corresponda como por ejemplo la siguiente figura.



Figura 10 - Espacio de la partición 1

En la figura se muestra como dentro del espacio reservado de Inodos se escribieron múltiples estructuras de inodos los cuales se entrar a detalle a continuación de dichas estructuras.

Nueva estructura

Journal

Esta viene siendo la nueva estructura a agregar en este proyecto, siendo una bitácora de todas las acciones que se realizan en el sistema de archivos logueado, este cuenta con la siguiente estructura:

NOMBRE	TIPO	DESCRIPCION
j_count	int	Lleva el conteo del journal que es.
j_content	information	Contiene toda la información de la acción que se hizo

Information

Este es el contenido que va a llevar el journal, donde se cuenta tanto con la operación realizada como la fecha en la que se realizó; está compuesto de la siguiente manera:

NOMBRE	TIPO	DESCRIPCION
i_operation	char[10]	Contiene la operación que se realizó.
i_path	char[32]	Contiene el path donde se realizó la operación.
i_content	char[64]	Contiene todo el contenido (si es un archivo.)
i_date	float	Contiene la fecha en la que se hizo la operación.

Nuevos comandos que se deben agregar al Sistema de Archivos

En el Proyecto 1, actualmente se cuenta con una serie de comandos iniciales que permiten la ejecución de tareas básicas. Sin embargo, para mejorar la funcionalidad y hacer el proyecto más completo, se requiere la implementación de nuevas funciones. Para ello, es necesario desarrollar y agregar nuevos comandos que amplíen las capacidades del sistema.

Se solicita que se implementen las siguientes funciones, apoyándose en la creación de nuevos comandos o agregar ciertos parámetros en comandos existentes:

1. FDISK (ADD y DELETE)

En este comando se agregan los parámetros delete y add para tener las funcionalidades de agregar, quitar espacio o eliminar particiones.

Parámetro	Categoría	Descripción
-size	Obligatorio al crear	Este parámetro recibirá un número que indicará el tamaño de la partición a crear. Debe ser positivo y mayor a cero, de lo contrario se mostrará un mensaje de error.
-unit	Opcional	Este parámetro recibirá una letra que indicará las unidades que utilizará el parámetro size. Podrá tener los siguientes valores: B: indicará que se utilizarán bytes. K: indicará que se utilizarán Kilobytes(1024 bytes) M: indicará que se utilizarán Megabytes(1024 * 1024 bytes). Este parámetro es opcional, si no se encuentra se creará una partición en Kilobytes. Si se utiliza un valor diferente mostrará un mensaje de error.
-path	Obligatorio	Este parámetro será la ruta en la que se encuentra el disco en el que se creará la partición. Este archivo ya debe existir, si no se mostrará un error.

-type	Opcional	Indicará que tipo de partición se creará. Ya que es opcional, se tomará como primaria en caso de que no se indique. Podrá tener los siguientes valores: P: en este caso se creará una partición primaria. E: en este caso se creará una partición extendida. L: Con este valor se creará una partición lógica. Las particiones lógicas sólo pueden estar dentro de la extendida sin sobrepasar su tamaño. Deberá tener en cuenta las restricciones de teoría de particiones: La suma de primarias y extendidas debe ser como máximo 4. Solo puede haber una partición extendida por disco. No se puede crear una partición lógica si no hay una extendida. Si se utiliza otro valor diferente a los anteriores deberá mostrar un mensaje de error.
-fit	Opcional	Indicará el ajuste que utilizará la partición para asignar espacio. Podrá tener los siguientes valores: BF: Indicará el mejor ajuste (Best Fit) FF: Utilizará el primer ajuste (First Fit) WF: Utilizará el peor ajuste (Worst Fit) Ya que es opcional, se tomará el peor ajuste si no está especificado en el comando. Si se utiliza otro valor que no sea alguno de los anteriores mostrará un mensaje de error.

-delete	Opcional	Este parámetro indica que se eliminará una partición. Este parámetro se utiliza junto con -name y -path. Se deberá mostrar un mensaje que permita confirmar la eliminación de dicha partición. Si la partición no existe deberá mostrar error. Si se elimina la partición extendida, deben eliminarse las particiones lógicas que tenga adentro. Recibirá los siguientes valores: Fast: Esta opción marca como vacío el espacio en la tabla de particiones. Full: Esta opción además marcar como vacío el espacio en la tabla de particiones, rellena el espacio con el carácter \0. Si se utiliza otro valor diferente, mostrará un mensaje de error.
-name	Obligatorio	Indicará el nombre de la partición. El nombre no debe repetirse dentro de las particiones de cada disco. Si se va a eliminar, la partición ya debe existir, si no existe debe mostrar un mensaje de error.
-add	Opcional	Este parámetro se utilizará para agregar o quitar espacio de la partición. Puede ser positivo o negativo. Tomará el parámetro units para las unidades a agregar o eliminar. En el caso de agregar espacio, deberá comprobar que exista espacio libre después de la partición. En el caso de quitar espacio se debe comprobar que quede espacio en la partición (no espacio negativo).

Ejemplos:

#Elimina de forma rápida una partición llamada
Partición 1

```
fdisk -delete=fast -name="Particion1" -  
path=/home/Disco1.dk
```

#Elimina de forma completa una partición llamada

Partición 1

```
fdisk -name=Particion1 -delete=full -  
path=/home/Disco1.dk
```

```
#Quitan 500 Kb de Partición 4 en Disco4.dk  
#Ignora los demás parámetros (size)  
#Se toma como válido el primero que aparezca, en  
este caso add
```

```
fdisk -add=-500 -size=10 -unit=K -  
path="/home/misdiscos/Disco4.dk"  
-name=Particion4
```

```
#Agrega 1 Mb a la partición Partición 4 del  
Disco4.dk  
#Se debe validar que haya espacio libre después de  
la partición
```

```
fdisk -add=1 -unit=M -path="/home/mis  
discos/Disco4.dk" -name="Particion 4"
```

2. UNMOUNT

Desmonta una partición del sistema. Se utilizará el id que se le asignó a la partición al momento de cargarla. Recibirá los siguientes parámetros:

Parámetro	Categoría	Descripción
-id	Obligatorio	Especifica el id de la partición que se desmontará. Si no existe el id deberá mostrar un error.

Ejemplos:

```
#Desmonta la partición con id 341A (En Disco1.dk)
```

```
umount -id=341
```

```
#Si no existe, se debe mostrar error umount -id=341A
```

Observaciones:

- En este paso debe cambiar el valor del correlativo de la partición a su estado inicial que es 0.
- La particion ya no se debe listar al momento de ejecutar el comando mounted

3. MKFS (FS)

Este comando realiza un formateo completo de la partición ya sea en EXT2 y EXT3, se formatea como ext2 por defecto si en caso el parámetro fs no está definido.

Tendrá los siguientes parámetros:

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-id	Obligatorio	Indicará el id que se generó con el comando mount. Si no existe mostrará error. Se utilizará para saber la partición y el disco que se utilizará para hacer el sistema de archivos.
-type	Opcional	Indicará que tipo de formateo se realizará. Podrá tener los siguientes valores: Full: en este caso se realizará un formateo completo. Ya que es opcional, se tomará como un formateo completo si no se especifica esta opción.
-fs	Opcional	Indica el sistema de archivos a formatear. Podrá tener los siguientes valores: 2fs: Para el sistema EXT2 3fs: Para el sistema EXT3

		Por defecto será ext2 .
--	--	--------------------------------

Ejemplos:

```
#Realiza un formateo completo de la partición en el id 061A en  
ext2
```

```
mkfs -type=full -id=061A
```

```
#Realiza un formateo completo de la partición que ocupa el id  
062A en EXT3
```

```
mkfs -id=062A -fs=3fs
```

4. Remove

Este comando permitirá eliminar un archivo o carpeta y todo su contenido, si el usuario que actualmente está logueado tiene acceso al permiso de escritura sobre el archivo y en el caso de carpetas, eliminará todos los archivos o subcarpetas en los que el usuario tenga permiso de escritura. Si no pudo eliminar un archivo o subcarpeta dentro de la carpeta por permisos, no deberá eliminar nada dentro de esa carpeta ni la carpeta como tal.

Tendrá los siguientes parámetros:

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-path	Obligatorio	Este parámetro será la ruta del archivo o carpeta que se eliminará. Si lleva espacios en blanco deberá encerrarse entre comillas. Si no existe el archivo o no tiene permisos de escritura en la carpeta o en el archivo, debe mostrarse un mensaje de error. Si no pudo eliminar algún archivo o carpeta no deberá eliminar los padres.

Ejemplos:

```
#Elimina el archivo a.txt, b.txt muestra error si no tiene
permiso
remove -path=/home/user/docs/a.txt
remove-path=/home/user/docs/b.txt

#Error por permisos
#Elimina la carpeta user y todo su contenido (docs, a.txt)
#Si el usuario no tuviera permiso de escritura sobre b.txt
#No debería eliminar las carpetas padres docs ni user, solo
a.txt
remove -Path=/home/user
remove -Path=/home/user
```

5. Rename

Este comando permitirá cambiar el nombre de un archivo o carpeta, si el usuario actualmente logueado tiene permiso de escritura sobre el archivo o carpeta.

Tendrá los siguientes parámetros:

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-path	Obligatorio	Este parámetro será la ruta del archivo o carpeta al que se le cambiará el nombre. Si lleva espacios en blanco deberá encerrarse entre comillas. Si no existe el archivo o carpeta o no tiene permisos de escritura deberá mostrar un mensaje de error.
-name	Obligatorio	Especificará el nuevo nombre del archivo, debe verificar que no exista un archivo con el mismo nombre, de ser así debe mostrar un mensaje de error.

Ejemplos:

```
#Cambia el nombre del archivo a.txt a b1.txt
rename -path=/home/user/docs/a.txt -name=b1.txt
```

```
#Deberá mostrar error ya que el archivo b1.txt ya existe
```

```
rename -path=/home/user/docs/c.txt -name=b1.txt
```

6. Copy

Este comando permitirá realizar una copia del archivo o carpeta y todo su contenido hacia otro destino. Tendrá los siguientes parámetros:

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-path	Obligatorio	Este parámetro será la ruta del archivo o carpeta que se desea copiar. Si lleva espacios en blanco deberá encerrarse entre comillas. Debe copiar todos los archivos y carpetas con todo su contenido, a los cuales tenga permiso de lectura. Si no tiene permiso de lectura, no realiza la copia únicamente de ese archivo o carpeta. Muestra un error si no existe la ruta
-destino	Obligatorio	Este parámetro será la ruta a donde se va a copiar el contenido. Debe tener permisos de escritura sobre esta carpeta, si no deberá mostrar un mensaje de error. De no existir la carpeta deberá mostrar un mensaje de error.

Ejemplos:

```
#/  
# home #664  
#user #664  
#documents #664  
#a.txt #664  
#b.txt #224  
#images #664  
#Copia documents a images  
copy -path="/home/user/documents" -destino="/home/images"  
# b.txt no se copia debido a falta de permisos #/  
# home #664  
#user #664  
#documents #664  
#a.txt #664  
#b.txt #224  
#images #664  
#documents #664  
#a.txt #664
```

7. Move

Este comando moverá un archivo o carpeta y todo su contenido hacia otro destino. Si el origen y destino están dentro de la misma partición, solo cambiará las referencias, para que ya no tenga el padre origen sino, el padre destino, y que los padres de la carpeta o archivo ya no tengan como hijo a la carpeta o archivo que se movió. Solo se deberán verificar los permisos de escritura sobre la carpeta o archivo origen. Tendrá los siguientes parámetros:

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-path	Obligatorio	Este parámetro será la ruta del archivo o carpeta que se desea mover. Si lleva espacios en blanco deberá encerrarse entre comillas. Debe mover todos los archivos y carpetas con todo su contenido, a los cuales tenga permiso de escritura. Si no tiene permiso de escritura, no realiza el movimiento Muestra un error si no existe la ruta.
- destino	Obligatorio	Este parámetro será la ruta de la carpeta a la que se moverá el archivo o carpeta. Debe tener permiso de escritura sobre la carpeta. Si lleva espacios en blanco deberá encerrarse entre comillas. Debe mostrar un mensaje de error si no tiene permisos para escribir o si la carpeta no existe.

Ejemplos:

```
#/  
# home #664  
# user #664  
# documents #664  
# a.txt #664  
# b.txt #224  
# images #664  
#Copia documents a images
```

```
move -path="/home/user/documents" -destino="/home/images"  
# mueve b.txt ya que solo se comprueban los permisos  
#/  
# home #664  
# user #664  
# images #664  
# documents #664  
# a.txt #664  
# b.txt #224
```

8. Find

Este comando permitirá realizar una búsqueda por el nombre del archivo o carpeta. Permitirá los siguientes caracteres especiales:

CARÁCTER	DESCRIPCIÓN
?	Un solo carácter
*	Uno o más caracteres

Recibe los siguientes parámetros:

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-path	Obligatorio	Este parámetro será la ruta de la carpeta en el que se inicia la búsqueda, deberá buscar en todo su contenido. Si lleva espacios en blanco deberá encerrarse entre comillas.
- name	Obligatorio	Debe tener permisos de lectura en los archivos que buscará. Indica el nombre de la carpeta o archivo que se desea buscar.

Ejemplos:

```
find -path = "/" -name=*  
# Arbol Actual  
# /  
#     |_ home                               #664  
#     |_ user                               #664  
#         |_ a.txt                         #664  
#         |_ b.txt                         #664  
#         |_ images                       #664  
#             |_ a.jpg                    #664  
#             |_ abcd.jpg                 #664
```

```
find -path = "/" -name = "?.*"  
# El resultado del comando sería  
# /  
#     |_ home  
#     |_ user  
#         |_ a.txt  
#         |_ b.txt  
#     |_ images  
#         |_ a.jpg
```

9. Chown

Cambiará el propietario de uno o varios archivos o carpetas. Lo podrá utilizar el usuario root en todos los archivos o carpetas y también lo podrán utilizar otros usuarios, pero solo sobre sus propios archivos. Recibirá los siguientes parámetros:

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-path	Obligatorio	Este parámetro será la ruta en la que se encuentra el archivo o carpeta a la que se le cambiará el propietario. Si no existe la ruta deberá mostrar mensaje de error
-r	opcional	Debe tener permisos de lectura en los archivos que buscará. Indica el nombre de la carpeta o archivo que se desea buscar.

-usuario	Obligatorio	Nombre del nuevo propietario, si no existe debe mostrar error
----------	-------------	---

Ejemplos:

#Cambia el propietario de la carpeta
home recursivamente

```
chown -path=/home -r -usuario=user2
```

#Cambia los
permisos de la
carpeta home

```
Chown -path=/home  
-usuario=user1
```

Pérdida del Sistema de Archivos EXT3

Simulate System Loss

Este formatea los siguientes bloques de datos para simular un fallo en el disco (una partición en específica), una inconsistencia o pérdida de información. Se deberán limpiar los siguientes bloques con el carácter /0.

- Bloque de bitmap de Inodos
- Bloque de bitmap de Bloques
- Área de Inodos
- Área de Bloques.

Ejemplo:

#Simulando la pérdida del sistema de archivos EXT3 en una partición

```
loss -id=061Disco1
```

- Se debe mostrar los reportes de bitmap antes de la ejecución y después. Se recomienda añadir la opción para visualizarlo desde el frontend

Journaling

Este mostrará la información de todas las transacciones realizadas mostrando la operación, la ruta, contenido, fecha y hora. Debe existir un apartado dentro de la interfaz gráfica para visualizarlo.

PARÁMETRO	CATEGORÍA	DESCRIPCIÓN
-id	Obligatorio	Indica el id de la partición que se utilizará.

#

Journaling en una partición
journaling -id=061Disco1

Operación	Path	Contenido	Fecha
mkdir	/	—	07/04/2025 19:07
mkfile	/ejemplo.txt	12345678901234567890	07/04/2025 19:07

Esta es solo la estructura que deberá llevar este reporte, pero queda a discreción del estudiante como mostrarlo por medio de la interfaz gráfica (se debe mostrar en la interfaz, no generar un reporte como el proyecto anterior).

Documentación

La documentación del proyecto consistirá en un **Manual Técnico y Manual de usuario**.

El **manual técnico** brindará una visión completa del funcionamiento interno y uso del sistema de archivos EXT3 simulado en esta aplicación web. También deberá incluir una explicación de todo el despliegue que se realizó en AWS. Este manual deberá incluir los siguientes componentes:

- **Descripción de la Arquitectura del Sistema:** Una explicación detallada de la estructura y conexión entre los módulos frontend y backend, apoyada por diagramas que muestran cómo se integran y comunican estos componentes. **En esta explicación también deberá entrar el despliegue a AWS junto con su arquitectura.**
- **Explicación de las Estructuras de Datos:** Descripción de las estructuras de datos fundamentales utilizadas en el sistema, tales como el MBR, EBR, inodos y bloques. Este apartado debe aclarar la función de cada estructura dentro del sistema de archivos y cómo se organizan y gestionan dentro del archivo binario .mia.
- **Descripción de los Comandos Implementados:** Un listado de todos los comandos disponibles en la aplicación (por ejemplo, MOVE, REMOVE, UNMOUNT), junto con una descripción detallada de cada uno. Esto incluye ejemplos de uso, parámetros requeridos y una explicación de sus efectos específicos sobre las estructuras internas del sistema de archivos.

Este **Manual Técnico** brindará una guía clara para el diseño y operación del sistema de archivos EXT3 en la aplicación web, además de ofrecer una referencia detallada para el uso de los comandos y la visualización de sus efectos en el sistema.

El manual de usuario debe contener una guía de cómo utilizar la página web y los comandos disponibles. Es necesario incluir capturas de pantalla de cada paso

4.2 Alcance del proyecto

Alcance obligatorio:

- Aplicación web funcional desplegada en AWS.
- Frontend alojado en un bucket S3.
- Backend desarrollado en C++ y desplegado en una instancia EC2.
- Área de entrada para ejecución de comandos.
- Área de salida para mostrar respuestas del sistema.
- Carga y ejecución de scripts con extensión **.smia**.
- Inicio y cierre de sesión mediante interfaz gráfica.
- Visualizador del sistema de archivos en modo lectura.
- Selección visual de discos creados.

- Selección visual de particiones.
- Navegación desde la carpeta raíz /.
- Visualización de carpetas, archivos, permisos e información básica.
- Visualización del contenido de archivos de texto.
- Implementación de estructuras EXT2 y EXT3.
- Implementación de journaling para EXT3.
- Implementación o adaptación de comandos principales.
- Documentación técnica y manual de usuario.
- Repositorio privado en GitHub con acceso habilitado para revisión

Alcance opcional:

- Mejoras visuales en la interfaz gráfica.
- Historial gráfico de operaciones realizadas.
- Visualización detallada del journaling.
- Validaciones adicionales de permisos y usuarios.

4.3 Recursos y herramientas a utilizar

Tipo (Obligatorio / opcional)	Categoría (Software / hardware / Plataforma / Etc)	Descripción
Tipo	Categoría	Descripción
Obligatorio	Lenguaje de programación	C++ para la implementación del backend y lógica del sistema de archivos.
Obligatorio	Plataforma cloud	AWS EC2 para desplegar el backend.
Obligatorio	Plataforma cloud	AWS S3 para desplegar el frontend como sitio web estático.
Obligatorio	Sistema operativo	Linux, preferiblemente Ubuntu, para la instancia EC2.
Obligatorio	Frontend	Framework moderno como React, Angular, Vue u otro equivalente.

4.4 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción
Sitio Web Funcional EN LA NUBE	Aplicación web ejecutable localmente que permite interactuar con el sistema de archivos simulado, ejecutar comandos y visualizar resultados. Debe estar en la AWS. • Frontend S3 • Backend EC2
Visualización	Visualización de discos, particiones, carpetas y archivos desde el frontend
Comandos Implementados	Implementación de la mayoría de los comandos definidos en el enunciado (especialmente aquellos relacionados a discos, usuarios y archivos).
Comentarios y Scripts	Soporte para ejecutar scripts <code>.smia</code> con múltiples comandos y mostrar comentarios dentro del área de salida de comandos.
Documentación Técnica	Manual que incluya la arquitectura del sistema, explicación de las estructuras implementadas y ejemplos de uso de los comandos.
Repositorio GitHub	Proyecto completo subido a un repositorio privado de GitHub, con acceso habilitado para los auxiliares, siguiendo el formato requerido.
Manual de Usuario	Documento que explica cómo usar el sistema desarrollado, incluyendo capturas de pantalla, pasos detallados y resolución de problemas comunes.

Nombre del repositorio: **MIA_2S2026_P2_carnet**

Usuarios de github de los auxiliares de cada sección:

1. **Sección A:** JSamRojas
2. **Sección B:** BramlleyProgram
3. **Sección C:** gkruiz
4. **Sección D:** nxz7

5. Material de apoyo

Categoría (Manual / Documentación oficial/ Ejemplo / Etc)	Link	Descripción
Repositorio del curso		Repositorio del curso con ejemplos orientados a la resolución del proyecto
Manual	https://github.com/nxz7/Ejemplos_MIA_S12026/blob/main/Semana12/Manual.md	Manual para subir a la nube aws un frontend en S3 y un backend en una instancia EC2
Documentación oficial	AWS EC2	Material de apoyo para configurar y desplegar el backend en una instancia Linux.
Documentación oficial	AWS S3	Material de apoyo para alojar el frontend como sitio web estático.
Ejemplo	https://youtu.be/SgSnz7kW-Ko	Ejemplo para conectar y configurar un backend y frontend en la nube AWS

**El link es opcional para una mejor guía del material de apoyo

6. Metodología

A continuación se presenta una metodología de la lógica que puedes seguir para elaborar el presente proyecto:

1. Analizar el Proyecto 1

Revisar los comandos, estructuras y funcionalidades ya implementadas para identificar qué partes pueden reutilizarse o adaptarse.

2. Diseñar la arquitectura del sistema

Definir la comunicación entre frontend, backend y archivos binarios `.mia`, considerando el despliegue en AWS.

3. Implementar o adaptar el backend en C++

Desarrollar la lógica de comandos, administración de discos, particiones, usuarios, permisos, archivos y carpetas.

4. Agregar soporte para EXT3

Implementar la estructura de journaling y adaptar el comando `mkfs` para soportar `2fs` y `3fs`.

5. Implementar los nuevos comandos solicitados

Agregar funcionalidades como `remove`, `rename`, `copy`, `move`, `find`, `chown`, `chmod`, `umount` y parámetros adicionales de `fdisk`.

6. Adaptar el frontend web

Adaptar la interfaz web para que permita la visualización de discos, particiones, carpetas y archivos.

7. Desplegar en AWS

Publicar el backend en EC2 y el frontend en S3.

8. Realizar pruebas funcionales

Ejecutar scripts de prueba para validar comandos, permisos, sesiones, reportes y visualización.

9. Documentar el proyecto

Elaborar manual técnico, manual de usuario y evidencias de funcionamiento.

Recomendaciones

- Validar la visualización de discos, particiones, carpetas y archivos en el frontend en S3.
- Documentar errores conocidos y decisiones técnicas.
- Tomar en cuenta los permisos necesarios para realizar la conexión entre S3 y EC2
- Verificar que el proyecto funcione en la nube antes de la fecha de entrega.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	14/09/2026	14/09/2026
Fecha de entrega	13/10/2026	13/10/2026
Calificación		

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, debe cumplir con los requisitos que se indiquen en esta sección, de no cumplir usted tendrá una nota con valor de cero puntos.

Tema	Descripción	Cumple (Sí/No)
El proyecto NO debe estar local	Únicamente se calificarán proyecto en la nube. Siguiendo las instrucciones: - Frontend: S3 - Backend: EC2	
Cumplimiento de la tecnología establecida	El backend debe estar desarrollado exclusivamente en C++	
Uso de herramientas o frameworks requeridos	El frontend debe estar implementado con un framework moderno como React, Angular o Vue.js . Debe usarse Graphviz para la generación de reportes.	
Gestión y entregas del proyecto	Todas las entregas parciales deben haberse completado según el cronograma. El proyecto debe estar en un repositorio privado de GitHub , con acceso habilitado.	

Pruebas y funcionalidad mínima	El sistema debe simular correctamente los comandos básicos el sistema de archivos EXT2/EXT3	
--------------------------------	---	--

8.2 Detalle de la Calificación

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades					
1.1	Documentación técnica (Manual Técnico + Manual de Usuario; arquitectura, estructuras, comandos, despliegue AWS)	7	Descripción	La documentación está completa, clara y organizada: describe arquitectura (frontend/backend), estructuras, comandos y despliegue en AWS	La documentación es incompleta o confusa: omite arquitectura, estructuras, comandos o despliegue;	
			Rango del punteo	7 - 4 /	4 - 0/	
1.2	Preguntas relacionadas al proyecto	5	Descripción	Responde con precisión y coherencia, demostrando dominio del funcionamiento del sistema, permisos, comandos y arquitectura implementada.	Respuestas superficiales o incorrectas; evidencia del desconocimiento del funcionamiento	
			Rango del punteo	5 - 3 /	2 - 0/	

	Sub-Total de Puntos	12				
2	Conocimientos					
2.1	Parte 1: Comandos previos (MKFS, LOGIN, LOGOUT, MKDIR, MKFILE)	5	Descripción	Todos los comandos previos funcionan correctamente, con parámetros válidos, manejo de errores y resultados esperados.	Uno o más comandos fallan, no cumplen lo solicitado o no manejan correctamente errores/parámetros.	
			Rango del punteo	5 - 3 /	2 - 0/	
2.2	Infraestructura EC2 (backend en C++ + Linux + Security Groups)	10	Descripción	Backend en C++ desplegado y operando en EC2 con Linux; configuración de seguridad (Security Groups) correcta y servicio accesible según se requiere.	No está desplegado correctamente, no ejecuta, no es C++/Linux, o la configuración de seguridad impide el funcionamiento.	
			Rango del punteo	10 - 6 /	5 - 0/	
2.3	Almacenamiento y despliegue S3 (sitio web estático + permisos + enlace)	11	Descripción	Frontend desplegado en S3 como sitio web estático, con permisos adecuados y enlace funcional y accesible.	Frontend no carga, permisos incorrectos o enlace inexistente/no funcional.	
			Rango del punteo	10 - 6 /	5 - 0/	

2.4	Conexion funcional de Backend(EC2) con frontend(S3)	7	Descripción	Existe una conexion funcional entre el frontend y el backend en la nube. Siguiendo las indicaciones: Frontend (S3) Backend (EC2)	No existe una conexion funcional entre el frontend y el backend en la nube	
			Rango del punteo	7 - 3 /	3 - 0/	
2.5	Aplicación web (flujo GUI: iniciar sesión, seleccionar disco/partición, navegación y visualización)	17	Descripción	La GUI permite: iniciar/cerrar sesión, seleccionar disco y partición, navegar carpetas y visualizar archivos (incluyendo permisos/contenido cuando aplica) de forma estable e intuitiva.	La GUI es incompleta o inestable: falla el flujo (login, selección, navegación) o no muestra correctamente el sistema de archivos/archivos.	
			Rango del punteo	17 - 10 /	9 - 0/	
3	Parte 3: Nuevos comandos (FDISK add/delete, UNMOUNT, MKFS fs, REMOVE, RENAME, COPY, MOVE, FIND, CHOWN)	24	Descripción	Implementa correctamente la mayoría de comandos con validación de parámetros, manejo de permisos y comportamiento conforme al enunciado.	Comandos incompletos o con errores significativos: validación deficiente, permisos mal aplicados o comportamientos no acordes al enunciado.	
			Rango del	24 - 13 /	12 - 0/	

			punteo			
	Parte 4: Journaling (LOSS + JOURNALING visible en la interfaz)	14	Descripción	Registra transacciones y se visualiza en la interfaz; simulación de pérdida (LOSS) funciona según especificación.	No registra/visualiza correctamente el journaling o la simulación de pérdida es incorrecta o inexistente.	
			Rango del punteo	14 - 8 /	7 - 0/	
	Sub-Total de Puntos	88				
	Total	100				

8.3 Comentarios Generales
